

Practical 8

Problem definition:

You are a Data Analytics Engineer at FinEdge Securities Ltd., a financial services company that provides stock market insights to investors. The company continuously receives stock price updates from multiple exchanges. Investors want to understand stock performance trends and identify the best investment opportunities. Traditional data processing systems struggle to handle the continuously growing volume of financial data.

Objectives:

1. To create a Spark Session for processing stock market data.
2. To load the stock dataset into a Spark DataFrame.
3. To display the stock records and dataset schema.
4. To group stock records based on stock symbols.
5. To calculate the average stock price for each stock.
6. To calculate the maximum and minimum stock prices.
7. To generate a stock summary report.
8. To identify stocks with higher observed average prices in the given dataset.
9. To interpret stock market trends using the analyzed data.
10. To generate analytical investment recommendations based on the available dataset.

Observation / Output:

Task 1: Create Spark Session. Load stock dataset into Spark DataFrame. Display schema and stock records. Group records based on stock symbol.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Stock Market Analysis") \
    .master("local[*]") \
    .getOrCreate()

print("Spark Session Created Successfully")
df = spark.read.csv(
    "stock_data.csv",
    header=True,
    inferSchema=True
)

print("Stock Records:")
df.show(10)

print("Schema of Stock Dataset:")
df.printSchema()

grouped_stocks = df.groupBy("Stock").count().orderBy("Stock")

grouped_stocks.show(truncate=False)
```

spark.stop()

```
===== STOCK MARKET ANALYSIS =====
Spark Session Created Successfully

===== STOCK RECORDS =====
+-----+-----+-----+
|Timestamp|Stock|Price|
+-----+-----+-----+
|2026-09-01|TCS|4100|
|2026-09-02|TCS|4140|
|2026-09-03|TCS|4125|
|2026-09-04|TCS|4180|
|2026-09-05|TCS|4210|
|2026-09-06|TCS|4195|
|2026-09-07|TCS|4250|
|2026-09-08|TCS|4280|
|2026-09-09|TCS|4310|
|2026-09-10|TCS|4350|
+-----+-----+-----+
only showing top 10 rows

===== SCHEMA OF STOCK DATASET =====
root
 |-- Timestamp: date (nullable = true)
 |-- Stock: string (nullable = true)
 |-- Price: integer (nullable = true)

===== STOCKS GROUPED BY SYMBOL =====
+-----+-----+
|Stock|count|
+-----+-----+
|HDFCBANK|10|
|INFY|10|
|ITC|10|
|RELIANCE|10|
|TCS|10|
+-----+-----+
```

Task 2: Calculate average stock price. Calculate maximum stock price. Calculate minimum stock price. Generate stock summary report.

```
stock_summary = df.groupBy("Stock").agg(
    round(avg("Price"), 2).alias("Average_Price"),
    max("Price").alias("Maximum_Price"),
    min("Price").alias("Minimum_Price")
).orderBy("Stock")
```

```
stock_summary.show(truncate=False)
```

```
|2026-09-07|TCS|4250|
|2026-09-08|TCS|4280|
|2026-09-09|TCS|4310|
|2026-09-10|TCS|4350|
+-----+-----+
only showing top 10 rows

===== SCHEMA OF STOCK DATASET =====
root
 |-- Timestamp: date (nullable = true)
 |-- Stock: string (nullable = true)
 |-- Price: integer (nullable = true)

===== STOCKS GROUPED BY SYMBOL =====
+-----+-----+
|Stock|count|
+-----+-----+
|HDFCBANK|10|
|INFY|10|
|ITC|10|
|RELIANCE|10|
|TCS|10|
+-----+-----+

===== STOCK SUMMARY REPORT =====
+-----+-----+-----+-----+
|Stock|Average_Price|Maximum_Price|Minimum_Price|
+-----+-----+-----+-----+
|HDFCBANK|1733.0|1805|1675|
|INFY|1916.0|1990|1850|
|ITC|495.4|515|480|
|RELIANCE|1490.0|1580|1420|
|TCS|4214.0|4350|4100|
+-----+-----+-----+-----+

(tushar@Tushar-laptop)~/Desktop/Practical8_BDA
$
```

Task 3: Identify best-performing stocks. Interpret market trends.

```
window_spec = Window.partitionBy("Stock").orderBy("Timestamp")
full_window = window_spec.rowsBetween(
    Window.unboundedPreceding,
    Window.unboundedFollowing
)
```

```
performance = df \
    .withColumn("Starting_Price", first("Price").over(full_window)) \
    .withColumn("Ending_Price", last("Price").over(full_window)) \
    .select("Stock", "Starting_Price", "Ending_Price") \
    .distinct() \
    .withColumn(
        "Return_Percentage",
        round(
            (col("Ending_Price") - col("Starting_Price"))
            / col("Starting_Price") * 100,
            2
        )
    ) \
    .orderBy(col("Return_Percentage").desc())
```

```
performance.show(truncate=False)
```

```
for row in performance.collect():
    if row["Return_Percentage"] > 0:
        trend = "Upward"
    elif row["Return_Percentage"] < 0:
        trend = "Downward"
    else:
        trend = "Stable"

    print(
        f"{row['Stock']}: {trend} trend, "
        f"Return = {row['Return_Percentage']}%"
    )
```

```
===== BEST-PERFORMING STOCKS =====
+-----+-----+-----+-----+
|Stock|Starting_Price|Ending_Price|Return_Percentage|
+-----+-----+-----+-----+
|RELIANCE|1420|1580|11.27|
|INFY|1850|1990|7.57|
|HDFCBANK|1600|1805|7.44|
|ITC|480|515|7.29|
|TCS|4100|4350|6.1|
+-----+-----+-----+-----+

===== MARKET TREND INTERPRETATION =====
RELIANCE: Upward trend, Return = 11.27%
INFY: Upward trend, Return = 7.57%
HDFCBANK: Upward trend, Return = 7.44%
ITC: Upward trend, Return = 7.29%
TCS: Upward trend, Return = 6.1%

(tushar@Tushar-laptop) ~/Desktop/Practical8_BDA
$
```

Task 4: Generate investment recommendations.

```
for row in performance.collect():
    if row["Return_Percentage"] >= 10:
        recommendation = "Strong positive performance in the sample period"
```

```

elif row["Return_Percentage"] >= 5:
    recommendation = "Positive performance in the sample period"
else:
    recommendation = "Moderate performance in the sample period"

print(
    f"{row['Stock']}: {recommendation} "
    f"(Return = {row['Return_Percentage']})%"
)

print("\nNote: These observations are based only on the sample dataset "
      "and should not be treated as real-world investment advice.")

```

```

===== INVESTMENT RECOMMENDATIONS =====
RELIANCE: Strong positive performance in the sample period (Return = 11.27%)
INFY: Positive performance in the sample period (Return = 7.57%)
HDFCBANK: Positive performance in the sample period (Return = 7.44%)
ITC: Positive performance in the sample period (Return = 7.29%)
TCS: Positive performance in the sample period (Return = 6.1%)

Note: These observations are based only on the sample dataset and should not be treated as real-world investment advice.

(tushar@Tushar-Laptop) ~/Desktop/Practical8_BDA
$

```

Key Questions /Analysis:

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process large volumes of data efficiently. It supports distributed data processing across multiple computing resources. Spark DataFrames make it easy to perform filtering, grouping, and aggregation operations. It can calculate financial metrics such as average, maximum, minimum, and returns quickly. Spark can process both historical and continuously arriving financial data. It also supports advanced analytics and machine learning operations. Therefore, Spark is useful for scalable and fast financial data analysis.

Q2. What is the purpose of grouping stock data by stock symbol?

Grouping stock data by stock symbol separates the records of different companies. It allows each stock to be analyzed independently. After grouping, statistics such as average, maximum, and minimum price can be calculated. It helps compare the performance of different stocks over the same period. Grouping also makes it easier to identify price trends for individual companies. It can be used to generate stock-wise summary reports. Thus, grouping provides a clear view of the performance of each stock.

Q3. How does average stock price help investors?

Average stock price represents the typical price level of a stock during a given period. It reduces the effect of individual daily price fluctuations. It helps investors understand the general price level of a stock. Average prices can be compared between different stocks or time periods. Changes in the average price can indicate a change in the overall price trend. It can also be used as a basic measure for further financial analysis. However, average price alone is not sufficient for making an investment decision.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the price range of a stock during a given period. The maximum price represents the highest observed price in the dataset. The minimum price represents the lowest observed price. These values help identify significant price movements.

The difference between them provides an indication of the stock's price variation. They can help analysts study potential volatility and market behavior. Therefore, maximum and minimum prices are useful for understanding price fluctuations.

Q5. How can Spark support real-time stock market monitoring?

Spark can process large amounts of stock market data efficiently. Its Structured Streaming capability can process continuously arriving data. New stock prices can be analyzed as they are received. Spark can calculate metrics such as returns, averages, and price changes continuously. It can also detect unusual movements and generate alerts based on defined conditions. The processed results can be connected to dashboards for monitoring. Thus, Spark can support scalable and near-real-time financial data analysis.

Supplementary Problems:

1. Calculate daily stock returns. Identify top-performing stock over a period. Compute moving average for stock prices. Compare stock volatility across companies. Visualize stock price trends using charts.

```
===== STOCK MARKET ANALYSIS =====
Spark Session Created Successfully

===== STOCK RECORDS =====
+-----+-----+-----+
|Timestamp|Stock|Price|
+-----+-----+-----+
|2026-09-01|TCS|4100|
|2026-09-02|TCS|4140|
|2026-09-03|TCS|4125|
|2026-09-04|TCS|4180|
|2026-09-05|TCS|4210|
|2026-09-06|TCS|4195|
|2026-09-07|TCS|4250|
|2026-09-08|TCS|4280|
|2026-09-09|TCS|4310|
|2026-09-10|TCS|4350|
+-----+-----+-----+
only showing top 10 rows

===== SCHEMA OF STOCK DATASET =====
root
 |-- Timestamp: date (nullable = true)
 |-- Stock: string (nullable = true)
 |-- Price: integer (nullable = true)

===== STOCKS GROUPED BY SYMBOL =====
+-----+-----+
|Stock|count|
+-----+-----+
|HDFCBANK|10|
|INFY|10|
|ITC|10|
|RELIANCE|10|
|TCS|10|
+-----+-----+
```

```
===== STOCK SUMMARY REPORT =====
+-----+-----+-----+-----+
|Stock|Average_Price|Maximum_Price|Minimum_Price|
+-----+-----+-----+-----+
|HDFCBANK|1733.0|1805|1675|
|INFY|1916.0|1990|1850|
|ITC|495.4|515|480|
|RELIANCE|1490.0|1580|1420|
|TCS|4214.0|4350|4100|
+-----+-----+-----+-----+

===== BEST-PERFORMING STOCKS =====
+-----+-----+-----+-----+
|Stock|Starting_Price|Ending_Price|Return_Percentage|
+-----+-----+-----+-----+
|RELIANCE|1420|1580|11.27|
|INFY|1850|1990|7.57|
|HDFCBANK|1680|1805|7.44|
|ITC|480|515|7.29|
|TCS|4100|4350|6.1|
+-----+-----+-----+-----+

===== MARKET TREND INTERPRETATION =====
RELIANCE: Upward trend, Return = 11.27%
INFY: Upward trend, Return = 7.57%
HDFCBANK: Upward trend, Return = 7.44%
ITC: Upward trend, Return = 7.29%
TCS: Upward trend, Return = 6.1%

===== INVESTMENT RECOMMENDATIONS =====
RELIANCE: Strong positive performance in the sample period (Return = 11.27%)
INFY: Positive performance in the sample period (Return = 7.57%)
HDFCBANK: Positive performance in the sample period (Return = 7.44%)
ITC: Positive performance in the sample period (Return = 7.29%)
TCS: Positive performance in the sample period (Return = 6.1%)

Note: These observations are based only on the sample dataset and should not be treated as real-world investment advice.
```

```
tushar@Tushar-laptop: ~/Des
```

===== DAILY STOCK RETURNS =====

Timestamp	Stock	Price	Daily_Return_Percent
2026-09-01	HDFCBANK	1680	NULL
2026-09-02	HDFCBANK	1675	-0.3
2026-09-03	HDFCBANK	1700	1.49
2026-09-04	HDFCBANK	1725	1.47
2026-09-05	HDFCBANK	1710	-0.87
2026-09-06	HDFCBANK	1740	1.75
2026-09-07	HDFCBANK	1765	1.44
2026-09-08	HDFCBANK	1750	-0.85
2026-09-09	HDFCBANK	1780	1.71
2026-09-10	HDFCBANK	1805	1.4
2026-09-01	INFY	1850	NULL
2026-09-02	INFY	1875	1.35
2026-09-03	INFY	1860	-0.8
2026-09-04	INFY	1895	1.88
2026-09-05	INFY	1910	0.79
2026-09-06	INFY	1935	1.31
2026-09-07	INFY	1920	-0.78
2026-09-08	INFY	1950	1.56
2026-09-09	INFY	1975	1.28
2026-09-10	INFY	1990	0.76
2026-09-01	ITC	480	NULL
2026-09-02	ITC	485	1.04
2026-09-03	ITC	482	-0.62
2026-09-04	ITC	490	1.66
2026-09-05	ITC	495	1.02
2026-09-06	ITC	492	-0.61
2026-09-07	ITC	500	1.63
2026-09-08	ITC	505	1.0
2026-09-09	ITC	510	0.99
2026-09-10	ITC	515	0.98
2026-09-01	RELIANCE	1420	NULL
2026-09-02	RELIANCE	1440	1.41
2026-09-03	RELIANCE	1435	-0.35
2026-09-04	RELIANCE	1465	2.09
2026-09-05	RELIANCE	1490	1.71
2026-09-06	RELIANCE	1475	-1.01
2026-09-07	RELIANCE	1510	2.37

```
tushar@Tushar-laptop: ~/Des
```

===== 3-DAY MOVING AVERAGE =====

Timestamp	Stock	Price	Moving_Average_3_Day
2026-09-01	HDFCBANK	1680	1680.0
2026-09-02	HDFCBANK	1675	1677.5
2026-09-03	HDFCBANK	1700	1685.0
2026-09-04	HDFCBANK	1725	1700.0
2026-09-05	HDFCBANK	1710	1711.67
2026-09-06	HDFCBANK	1740	1725.0
2026-09-07	HDFCBANK	1765	1738.33
2026-09-08	HDFCBANK	1750	1751.67
2026-09-09	HDFCBANK	1780	1765.0
2026-09-10	HDFCBANK	1805	1778.33
2026-09-01	INFY	1850	1850.0
2026-09-02	INFY	1875	1862.5
2026-09-03	INFY	1860	1861.67
2026-09-04	INFY	1895	1876.67
2026-09-05	INFY	1910	1889.33
2026-09-06	INFY	1935	1912.33
2026-09-07	INFY	1920	1921.67
2026-09-08	INFY	1950	1935.0
2026-09-09	INFY	1975	1948.33
2026-09-10	INFY	1990	1971.67
2026-09-01	ITC	480	480.0
2026-09-02	ITC	485	482.5
2026-09-03	ITC	482	482.33
2026-09-04	ITC	490	485.67
2026-09-05	ITC	495	489.0
2026-09-06	ITC	492	492.33
2026-09-07	ITC	500	495.67
2026-09-08	ITC	505	499.0
2026-09-09	ITC	510	505.0
2026-09-10	ITC	515	510.0
2026-09-01	RELIANCE	1420	1420.0
2026-09-02	RELIANCE	1440	1430.0
2026-09-03	RELIANCE	1435	1431.67
2026-09-04	RELIANCE	1465	1446.67
2026-09-05	RELIANCE	1490	1463.33
2026-09-06	RELIANCE	1475	1476.67
2026-09-07	RELIANCE	1510	1491.67

```
tushar@Tushar-laptop: ~/Des
```

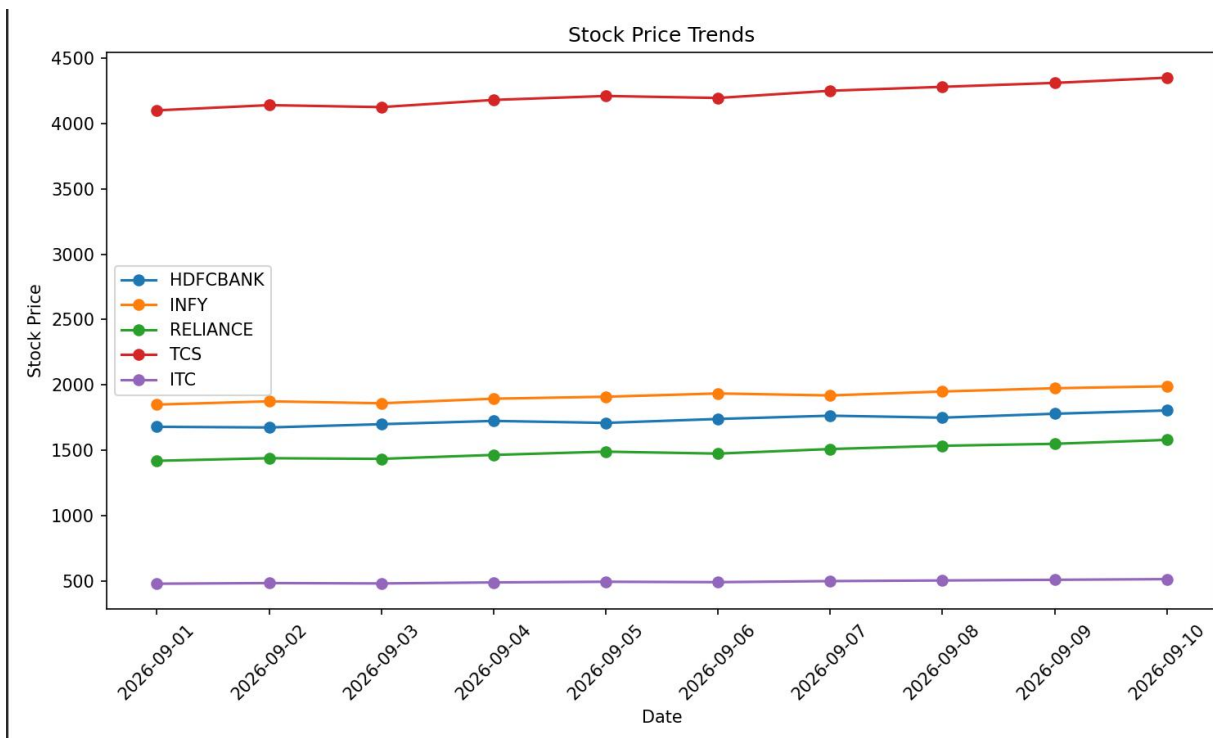
2026-09-08	ITC	505	499.0
2026-09-09	ITC	510	505.0
2026-09-10	ITC	515	510.0
2026-09-01	RELIANCE	1420	1420.0
2026-09-02	RELIANCE	1440	1430.0
2026-09-03	RELIANCE	1435	1431.67
2026-09-04	RELIANCE	1465	1446.67
2026-09-05	RELIANCE	1490	1463.33
2026-09-06	RELIANCE	1475	1476.67
2026-09-07	RELIANCE	1510	1491.67
2026-09-08	RELIANCE	1535	1506.67
2026-09-09	RELIANCE	1550	1531.67
2026-09-10	RELIANCE	1580	1555.0
2026-09-01	TCS	4100	4100.0
2026-09-02	TCS	4140	4120.0
2026-09-03	TCS	4125	4121.67
2026-09-04	TCS	4180	4148.33
2026-09-05	TCS	4210	4171.67
2026-09-06	TCS	4195	4195.0
2026-09-07	TCS	4250	4218.33
2026-09-08	TCS	4280	4241.67
2026-09-09	TCS	4310	4280.0
2026-09-10	TCS	4350	4313.33

===== STOCK VOLATILITY ANALYSIS =====

Stock	Return_Volatility
RELIANCE	1.1489
HDFCBANK	1.1262
INFY	0.9742
ITC	0.8401
TCS	0.626

===== STOCK PRICE TREND CHART =====

Chart saved as stock_price_trends.png



Tools / Technology used:

Apache Spark 3.x | PySpark | Python 3.11 | VS Code / Jupyter Notebook | Matplotlib
(Optional Visualization)

Conclusion:

The stock market analysis practical was successfully implemented using Apache Spark and PySpark DataFrames. A sample stock dataset containing daily prices of multiple companies was loaded and analyzed efficiently. Spark DataFrame operations were used to group stock records and calculate important measures such as average, maximum, and minimum stock prices. The analysis also helped identify stock performance and interpret overall market trends during the selected period. Additional analysis such as daily stock returns, moving averages, and volatility comparison provided deeper insights into stock price behavior. Finally, stock price trends were visualized using a line chart for easier comparison between companies. Overall, this practical demonstrated how Apache Spark can be used for large-scale financial data processing, aggregation, trend analysis, and business intelligence reporting, while showing how data-driven insights can support further financial analysis.

Github Repo:

<https://github.com/tushar-2606/BDA-SHP-2026>